ETSI TR 104 239-3 V1.1.1 (2026-09)



TECHNICAL REPORT

Cyber Security (CYBER);
Quantum-Safe Cryptography (QSC);
Secure Implementation Guidance for
Key Encapsulation Mechanisms and Digital Signature Schemes;
Part 3: ML-DSA

Reference
DTR/CYBER-QSC-0027-3

Keywords
cyber security, digital signature,
quantum safe cryptography

ETSI

650 Route des Lucioles
F-06921 Sophia Antipolis Cedex - FRANCE

Tel.: +33 4 92 94 42 00 Fax: +33 4 93 65 47 16

Siret N° 348 623 562 00017 - APE 7112B
Association à but non lucratif enregistrée à la
Sous-Préfecture de Grasse (06) N° w061004871

Important notice

The present document can be downloaded from the
ETSI Search & Browse Standards application.

The present document may be made available in electronic versions and/or in print. The content of any electronic and/or print versions of the present document shall not be modified without the prior written authorization of ETSI. In case of any existing or perceived difference in contents between such versions and/or in print, the prevailing version of an ETSI deliverable is the one made publicly available in PDF format on ETSI deliver repository.

Users should be aware that the present document may be revised or have its status changed,
this information is available in the Milestones listing.

If you find errors in the present document, please send your comments to
the relevant service listed under Committee Support Staff.

If you find a security vulnerability in the present document, please report it through our
Coordinated Vulnerability Disclosure (CVD) program.

Notice of disclaimer & limitation of liability

The information provided in the present deliverable is directed solely to professionals who have the appropriate degree of experience to understand and interpret its content in accordance with generally accepted engineering or other professional standard and applicable regulations.
No recommendation as to products and services or vendors is made or should be implied.
No representation or warranty is made that this deliverable is technically accurate or sufficient or conforms to any law and/or governmental rule and/or regulation and further, no representation or warranty is made of merchantability or fitness for any particular purpose or against infringement of intellectual property rights.
In no event shall ETSI be held liable for loss of profits or any other incidental or consequential damages.

Any software contained in this deliverable is provided "AS IS" with no warranties, express or implied, including but not limited to, the warranties of merchantability, fitness for a particular purpose and non-infringement of intellectual property rights and ETSI shall not be held liable in any event for any damages whatsoever (including, without limitation, damages for loss of profits, business interruption, loss of information, or any other pecuniary loss) arising out of or related to the use of or inability to use the software.

Copyright Notification

No part of this document may be reproduced in any form, by any means and in any media, without the prior written authorization of ETSI and except as expressly permitted below.
By way of exception and when the document is a normative deliverable (European Standard (EN), Technical Specification (TS), Group Specification (GS) or ETSI Standard (ES)), ETSI authorizes to reproduce and incorporate into products, services and technical documentation only those extracts (e.g. templates) that are strictly necessary for the technical implementation of the normative deliverable, to ensure compliance with the latter.
Nothing in this notice shall be construed as limiting any mandatory exceptions to copyright provided by applicable law.

© ETSI 2026.
All rights reserved.

Contents

Intellectual Property Rights 5
Foreword 5
Modal verbs terminology 5
1 Scope 6
2 References 6
2.1 Normative references 6
2.2 Informative references 6
3 Definition of terms, symbols and abbreviations 8
3.1 Terms 8
3.2 Symbols 8
3.3 Abbreviations 8
4 Introduction 9
5 ML-DSA interfaces 10
5.1 Parameters 10
5.2 Key generation 10
5.2.1 Key formats 10
5.2.2 Full key generation 11
5.2.3 Seeded key generation 11
5.3 Signature generation 11
5.3.1 Overview 11
5.3.2 Pure ML-DSA 12
5.3.3 HashML-DSA 13
5.3.4 External μ 13
5.4 Signature verification 15
5.4.1 Pure ML-DSA 15
5.4.2 HashML-DSA 15
5.4.3 External μ 16
6 General implementation considerations 17
6.1 Perform input validation 17
6.1.1 Signature generation inputs 17
6.1.2 Signature verification inputs 17
6.1.3 HashML-DSA inputs 18
6.1.4 External μ inputs 18
6.1.5 Internal algorithms 18
6.2 Perform output validation 19
6.2.1 Rejection sampling checks 19
6.2.2 Pairwise consistency test 19
6.2.3 Self testing 20
6.3 Prevent leakage of intermediate values 20
6.3.1 Zeroise intermediate values after use 20
6.3.2 Limit external access to internal functions 21
6.4 Handle errors gracefully 21
6.4.1 Specified errors 21
6.4.2 Unspecified errors 21
6.4.3 Verification failures 22
6.5 Randomness 22
7 Side-channel and fault attack considerations 22
7.1 Key Generation 22
7.1.1 Overview 22
7.1.2 Timing Analysis Considerations 23
7.1.3 Power Analysis Considerations 23
7.1.4 Fault Attack Considerations 24

7.2 Signature generation 24
7.2.1 Overview 24
7.2.2 Timing Analysis Considerations 25
7.2.3 Power Analysis Considerations 25
7.2.4 Fault Attack Considerations 26
7.3 Signature verification 26
7.3.1 Overview 26
7.3.2 Timing Analysis Considerations 26
7.3.3 Power Analysis Considerations 26
7.3.4 Fault Attack Considerations 26
8 Testing and formal verification considerations 27
8.1 Testing Considerations 27
8.2 Formal Verification Considerations 28
Annex A: NIST FIPS 204 algorithms 29
A.1 Pure digital signature (external) 29
A.2 Pre-hash digital signature (external) 29
A.3 De-randomized digital signature (internal only) 29
A.4 Sampling 29
A.5 Arithmetic 30
A.6 Encoding 30
Annex B: ML-DSA security properties 31
B.1 Unforgeability 31
B.2 Beyond unforgeability 32
Annex C: Hybrid ML-DSA 33
C.1 Deployment considerations 33
C.2 Implementation considerations 33
Annex D: Change history 35
History 36

Intellectual Property Rights

Essential patents

IPRs essential or potentially essential to normative deliverables (European Standard (EN), Technical Specification (TS), Group Specification (GS) or ETSI Standard (ES)) may have been declared to ETSI. The declarations pertaining to these essential IPRs, if any, are publicly available for **ETSI members and non-members**, and can be found in ETSI SR 000 314: *"Intellectual Property Rights (IPRs); Essential, or potentially Essential, IPRs notified to ETSI in respect of ETSI standards"*, which is available from the ETSI Secretariat. Latest updates are available on the ETSI IPR online database.

Pursuant to the ETSI Directives including the ETSI IPR Policy, no investigation regarding the essentiality of IPRs, including IPR searches, has been carried out by ETSI. No guarantee can be given as to the existence of other IPRs not referenced in ETSI SR 000 314 (or the updates on the ETSI Web server) which are, or may be, or may become, essential to the present document.

Trademarks

The present document may include trademarks and/or tradenames which are asserted and/or registered by their owners. ETSI claims no ownership of these except for any which are indicated as being the property of ETSI, and conveys no right to use or reproduce any trademark and/or tradename. Mention of those trademarks in the present document does not constitute an endorsement by ETSI of products, services or organizations associated with those trademarks.

DECT™, **PLUGTESTS™**, **UMTS™** and the ETSI logo are trademarks of ETSI registered for the benefit of its Members. **3GPP™**, **LTE™** and **5G™** logo are trademarks of ETSI registered for the benefit of its Members and of the 3GPP Organizational Partners. **oneM2M™** logo is a trademark of ETSI registered for the benefit of its Members and of the oneM2M Partners. **GSM**® and the GSM logo are trademarks registered and owned by the GSM Association.

Foreword

This Technical Report (TR) has been produced by ETSI Technical Committee Cyber Security (CYBER).

The present document is part 3 of a multi-part deliverable. Full details of the entire series can be found in part 1 [i.2].

The present document provides implementation guidance for the Module-Lattice-based Digital Signature Algorithm (ML-DSA).

Modal verbs terminology

In the present document "**should**", "**should not**", "**may**", "**need not**", "**will**", "**will not**", "**can**" and "**cannot**" are to be interpreted as described in clause 3.2 of the ETSI Drafting Rules (Verbal forms for the expression of provisions).

"**must**" and "**must not**" are **NOT** allowed in ETSI deliverables except when used in direct citation.

1 Scope

The present document provides developers with guidance to aid the secure implementation of the quantum-safe digital signature algorithm ML-DSA as specified by NIST FIPS 204 [i.7]. This highlights some potential ML-DSA implementation hazards; identifies some side-channel and fault attack issues specific to ML-DSA; considers some testing and formal verification aspects relevant to ML-DSA; and briefly discusses the secure usage of ML-DSA.

2 References

2.1 Normative references

Normative references are not applicable in the present document.

2.2 Informative references

References are either specific (identified by date of publication and/or edition number or version number) or non-specific. For specific references, only the cited version applies. For non-specific references, the latest version of the referenced document (including any amendments) applies.

NOTE: While any hyperlinks included in this clause were valid at the time of publication ETSI cannot guarantee their long-term validity.

The following referenced documents may be useful in implementing an ETSI deliverable or add to the reader's understanding, but are not required for conformance to the present document.

[i.1] ETSI TR 103 966: "CYBER Security (CYBER); Quantum-Safe Cryptography (QSC); Deployment Considerations for Hybrid Schemes".

[i.2] ETSI TR 104 239-1: "Cyber Security (CYBER); Quantum-Safe Cryptography (QSC); Secure Implementation Guidance for Key Encapsulation Mechanisms and Digital Signature Schemes; Part 1: General".

[i.3] IETF RFC 9881: "Internet X.509 Public Key Infrastructure - Algorithm identifiers for the Module-Lattice-based Digital Signature Algorithm".

[i.4] IRTF RFC 8032: "Edwards-curve Digital Signature Algorithm (EdDSA)".

[i.5] ISO/IEC 24759:2025: "Information security, cybersecurity and privacy protection - Test requirements for cryptographic modules".

[i.6] NIST FIPS 186-5: "Digital Signature Standard (DSS)".

[i.7] NIST FIPS 204: "Module-Lattice-Based Digital Signature Standard".

[i.8] NIST: "FAQ - FIPS 204 - Computing mu".

[i.9] NIST SP 800-57A Rev. 6 (Initial Public Draft): "Recommendation for Key Management: Part 1 - General".

[i.10] NIST SP 800-89: "Recommendation for Obtaining Assurances for Digital Signature Applications".

[i.11] NIST: "Cryptographic Algorithm Validation Program (CAVP)".

[i.12] NIST: "Automated Cryptographic Validation Test System".

[i.13] NIST: "Implementation Guidance for FIPS 140-3 and the Cryptographic Module Validation Program".

[i.14] S. Bai et al.: "CRYSTALS-Dilithium: Algorithm specifications and supporting documentation (Version 3.1)". NIST Post-Quantum Cryptography Standardization Process.

[i.15] S. Bauer and F. De Santis: "Forging Dilithium and Falcon signatures by single fault injection". FDTC 2023.

[i.16] B. Benčina et al.: "Post-quantum cryptographic analysis of SSH". S&P, 2025.

[i.17] F. Bergsma et al.: "Multi-ciphersuite security of the Secure Shell (SSH) protocol". ACM CCS, 2014.

[i.18] F. Bergsma et al.: "Multi-ciphersuite security of the Secure Shell (SSH) protocol". IACR ePrint 2013/813, June 5, 2020.

[i.19] D.J. Bernstein et al.: "KyberSlash: Exploiting secret-dependent division timings in Kyber implementations". CHES, 2025.

[i.20] J. Brendel et al.: "The provable security of Ed25519: Theory and practice". S&P, 2021.

[i.21] K. Chalkias, F. Garillot and V. Nikolaenko: "Taming the many EdDSAs". SSR, 2020.

[i.22] Community Cryptography Specification Project: "Project Wycheproof".

[i.23] Community Cryptography Specification Project: "Community Cryptography Test Vectors (CCTV)".

[i.24] J.-S. Coron et al.: "Improved gadgets for the high-order masking of Dilithium". CHES, 2023.

[i.25] J.-S. Coron et al.: "Improved high-order masked generation of masking vector and rejection sampling in Dilithium". CHES, 2024.

[i.26] C. Cremers et al.: "BUFFing signature schemes beyond unforgeability and the case of post-quantum signatures". S&P, 2021.

[i.27] Cryspen: "libcrux".

[i.28] Cybersecurity & Infrastructure Security Agency, US Government: "Secure-by-Design - Shifting the Balance of Cybersecurity Risk: Principles and Approaches for Secure by Design Software".

[i.29] C. Decker and R. Wattenhofer: "Bitcoin transaction malleability and MtGox". ESORICS, 2014.

[i.30] Department of Science, Innovation and Technology, UK Government: "Software Security Code of Practice".

[i.31] J. Don et al.: "On the (In)Security of the BUFF transform". CRYPTO, 2024.

[i.32] A. Fiat and A. Shamir: "How to prove yourself: Practical solutions to identification and signature problems". CRYPTO, 1986.

[i.33] B. Gierlichs et al.: "Mutual information analysis". CHES, 2008.

[i.34] T. Hollebeek, S. Schmieg and B. Westerbaan, "Use of ML-DSA in TLS 1.3 (work in progress)". IETF Internet-Draft, draft-ietf-tls-mldsa.

[i.35] K. Jackson, C. Miller and D. Wang: "Evaluating the security of CRYSTALS-Dilithium in the Quantum Random Oracle Model". EUROCRYPT, 2024.

[i.36] E. Karatsiolis et al.: "Public key linting for ML-KEM and ML-DSA". ACNS, 2025.

[i.37] E. Kiltz, V. Lyubashevsky and C. Schaffner: "A concrete treatment of Fiat-Shamir signatures in the Quantum Random-Oracle Model". EUROCRYPT, 2018.

[i.38] V. Lyubashevsky: "Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures". ASIACRYPT, 2009.

[i.39] Monero: "Disclosure of a Major Bug In Cryptonote Based Currencies".

[i.40] P.Q. Nguyen and O. Regev: "Learning a parallelpiped: Cryptanalysis of GGH and NTRU signatures". EUROCRYPT, 2006.

[i.41] M. Ounsworth et al, "Composite Module-Lattice-based Digital Signature Algorithm (ML-DSA) for use in X.509 Public Key Infrastructure (work in progress)". IETF Internet-Draft, draft-ietf-lamps-pq-composite-sigs.

[i.42] Post-Quantum Cryptography Alliance: "mldsa-native".

[i.43] Post-Quantum Cryptography Alliance: "mlkem-native".

[i.44] P. Ravi et al.: "Exploiting determinism in lattice-based signatures: Practical fault attacks on pqm4 implementations of NIST candidates". Asia CCS, 2019.

[i.45] P. Ravi et al.: "On configurable SCA countermeasures against single trace attacks for the NTT". SPACE, 2020.

[i.46] Sebastien Riou: "BTVG-MLDSA".

[i.47] M. Stevens, A. Lenstra and B. de Weger: "Chosen-prefix collisions for MD5 and colliding X.509 certificates for different identities". EUROCRYPT, 2007.

[i.48] Symbolic Software: "Crucible".

[i.49] Symbolic Software: "Even More Bugs in CE Labs' libcrux: ML-DSA".

[i.50] UK National Cyber Security Centre: "Next steps in preparing for post-quantum cryptography".

[i.51] R. Wang et al.: "Unpacking needs protection: A single-trace secret key recovery attack on Dilithium". IACR Communications in Cryptology, 2024.

[i.52] P. Wuille, J. Nick and T. Ruffing: "BIP 340: Schnorr Signatures for secp256k1".

3 Definition of terms, symbols and abbreviations

3.1 Terms

Void.

3.2 Symbols

For the purposes of the present document, the following symbols apply:

$\|a\|_\infty$	Maximum absolute value of a coefficient in a
$\mathrm{wt}(a)$	Number of non-zero coefficients in a
(a, b)	Ordered pair of values a and b
$a \mid\mid b$	Concatenation of the byte strings a and b
$a = b$	Return true if the values a and b are equal and false otherwise
$a \leftarrow b$	Assign the value b to the variable a

3.3 Abbreviations

For the purposes of the present document, the following abbreviations apply:

AES	Advanced Encryption Standard
BIP	Bitcoin Improvement Proposal
BUFF	Beyond Unforgeability
CPA	Correlational Power Analysis
CVE	Common Vulnerabilities and Exposures
DPA	Differential Power Analysis
ECDSA	Elliptic Curve Digital Signature Algorithm
EdDSA	Edwards-curve Digital Signature Algorithm

EUF-CMA	Existential Unforgeability against Chosen-Message Attacks
FIPS	Federal Information Processing Standard
ISO	International Organization for Standardization
LWE	Learning With Errors
MBS	Message-Bound Signatures
ML-DSA	Module-Lattice-based Digital Signature Algorithm
ML-KEM	Module-Lattice-based Key Encapsulation Mechanism
M-S-UEO	Malicious-Strong Universal Exclusive Ownership
NIST	National Institute of Standards and Technology
NR	Non Re-signability
NTT	Number Theoretic Transform
PKCS	Public Key Cryptography Standards
PQC	Post-Quantum Cryptography
RBG	Random Bit Generator
RSA	Rivest-Shamir-Adleman
SHA	Secure Hash Algorithm
SHAKE	Secure Hash Algorithm Keccak
SIS	Short Integer Solution
SP	Special Publication
SSH	Secure Shell
SUF-CMA	Strong Unforgeability against Chosen-Message Attacks
TLS	Transport Layer Security

4 Introduction

The Module-Lattice-based Digital Signature Algorithm (ML-DSA) is a post-quantum signature algorithm selected for standardization following the multi-year NIST Post-Quantum Cryptography (PQC) Standardization Process. It is specified in NIST FIPS 204 [i.7] and support for ML-DSA is being integrated into internet protocols such as TLS [i.34].

NOTE: ML-DSA is based on the CRYSTALS-Dilithium submission to the NIST PQC Process [i.14].

ML-DSA received a significant amount of analysis during the NIST PQC Process, but its security also depends on the way in which it is implemented and used. The present document builds on the general guidance in ETSI TR 104 239-1 [i.2] to provide developers with some specific guidance to aid the secure implementation of ML-DSA:

- It describes the ML-DSA interfaces, including the difference between the "pure" ML-DSA and HashML-DSA algorithms, and different options for private key formats (clause 5).
- It extends the general implementation considerations from ETSI TR 104 239-1 [i.2] with ML-DSA specific aspects such as input validation, consistency checks, and error handling (clause 6).
- It discusses different side-channel considerations for ML-DSA key generation, signature generation and signature verification (clause 7).
- It touches on testing and formal validation for ML-DSA implementations (clause 8).

The present document also gives some background on ML-DSA security properties (annex B) and some considerations on the use of ML-DSA in hybrid schemes (annex C).

As with ETSI TR 104 239-1 [i.2], the guidance is primarily aimed at secure implementations in software, although some of the recommendations will also be relevant to hardware implementations. Developers implementing ML-DSA should also ensure that they follow best practice for developing secure systems in general [i.28], [i.30].

5 ML-DSA interfaces

5.1 Parameters

NIST FIPS 204 [i.7] specifies three approved parameter sets for ML-DSA:

- **ML-DSA-44** targets security category 2, which is equivalent to finding collisions for SHA-256;
- **ML-DSA-65** targets security category 3, which is equivalent to key recovery for AES-192; and
- **ML-DSA-87** targets security category 5, which is equivalent to key recovery for AES-256.

NIST FIPS 204 [i.7] does not recommend a particular ML-DSA parameter set. However, ML-DSA-65 will be generally appropriate for use in most applications (see, for example, [i.50]). ML-DSA-44 can be used in applications that require smaller public keys or signatures. ML-DSA-87 can be used in applications that require a higher level of security.

Table 1 lists the sizes of the ML-DSA private signing keys, public verification keys and signatures.

Table 1: ML-DSA private key, public key and signature lengths (in bytes)

Parameter set	Private signing key	Public verification key	Signature
ML-DSA-44	2 560	1 312	2 420
ML-DSA-65	4 032	1 952	3 309
ML-DSA-87	4 896	2 592	4 627

5.2 Key generation

5.2.1 Key formats

The ML-DSA public verification key pk is a byte string that encodes a pair of values (ρ, t_1).

The ML-DSA private signing key sk is a byte string that encodes a 6-tuple of values (ρ, K, tr, s_1, s_2, t_0).

NOTE 1: The public seed ρ is contained in both the public verification key pk and the private signing key sk. It is not possible to recover the full public key directly from the private key. If an implementation of ML-DSA signature generation is intended to check that the signatures verify correctly before returning them (see clause 6.2.1), then it will need the public key as a separate input.

NOTE 2: The private signing key includes the hash tr of the public verification key pk. This is needed when computing the message representative μ (see clause 5.3.4).

NOTE 3: If an adversary is able to recover s_1 from the private signing key sk, then this would allow them to forge signatures for arbitrary messages [i.44].

NOTE 4: If an adversary is able to recover K from the private signing key sk, then with reasonable probability they can use this to recover s_1 from a valid message-signature pair (m, σ) provided that they also know, or can guess, the random value rnd used to generate the signature. This will be true for deterministic signing (see clause 5.3.1).

ML-DSA private signing keys are significantly larger than its public verification keys. Large private keys could be problematic for applications where secure key storage is limited or where there are many private signing keys.

Applications that are expected to support ML-DSA signature generation and verification with the same key pair will also need to store both the private and public keys since the public key cannot be recomputed from the private key.

Consequently, NIST FIPS 204 ([i.7], section 3.6.3) explicitly allows a 32-byte private seed to be stored and used in place of the full private signing key, provided that it is given the same level of protection as the private key.

5.2.2 Full key generation

The specification of ML-DSA key generation in NIST FIPS 204 [i.7] (algorithm 1) samples a random 32-byte private seed and deterministically generates the key pair from that seed via an internal key generation function (see Table 2).

Table 2: ML-DSA key generation interface

ML-DSA.KeyGen()		
Input:	None	
Output:	Public verification key $\boldsymbol{pk}$ and private signing key $\boldsymbol{sk}$, or Error	
1.	$\xi \leftarrow$ RBG(32)	
2.	if ξ = NULL then	
3.	return Error	(RBG failure)
4.	end if	
5.	$(pk, sk) \leftarrow$ ML-DSA.KeyGen_internal(ξ)	(Generate key pair from private seed)
6.	return (pk, sk)	

5.2.3 Seeded key generation

The change to ML-DSA key generation when using a private seed in place of the full private signing key is minimal (see Table 3).

Table 3: ML-DSA seed format key generation interface

ML-DSA.SeedKeyGen()		
Input:	None	
Output:	Public verification key $\boldsymbol{pk}$ and private seed $\boldsymbol{\xi}$, or Error	
1.	$\xi \leftarrow$ RBG(32)	
2.	if ξ = NULL then	
3.	return Error	(RBG failure)
4.	end if	
5.	$(pk, _) \leftarrow$ ML-DSA.KeyGen_internal(ξ)	(Expand public key from private seed)
6.	return (pk, ξ)	

The public verification key and private signing key can be recomputed from the private seed via the internal ML-DSA key generation algorithm (NIST FIPS 204 [i.7], algorithm 6).

It is generally not possible to recover the private seed from the private signing key. Some protocols therefore allow the private signing key, the private seed, or both to be used [i.3].

5.3 Signature generation

5.3.1 Overview

NIST FIPS 204 [i.7] specifies two versions of ML-DSA:

- Pure ML-DSA, where the message is passed directly to signature generation and verification; and
- HashML-DSA, where the message is hashed before input to signature generation and verification.

NOTE 1: Pure ML-DSA and HashML-DSA are considered to be different algorithms with different algorithm identifiers.

NOTE 2: Although key generation is the same for pure ML-DSA and HashML-DSA, NIST FIPS 204 [i.7] recommends against using the same key pair in both algorithms.

NOTE 3: An ML-DSA signature σ is a byte string that encodes a triple of values $(\tilde{c}, z, h)$. The format is the same for both pure ML-DSA and HashML-DSA.

The advantage of HashML-DSA is that when signature generation is being performed by a cryptographic module, only the hash of the message needs to be transferred to the module, which may be useful in cases where there are limitations to the amount of data that can be transferred to or processed by the module. However, HashML-DSA adds complexity as several different hash functions could be used to pre-hash the message.

Consequently, NIST FIPS 204 [i.7] also permits a version of pure ML-DSA which splits the computation of the message representative from the rest of the signing process (see NIST FIPS 204 [i.7], algorithm 7). The message representative can be computed outside the cryptographic module, since it does not involve the private signing key, and then passed to the module for signing instead of the full message. This technique is informally known as "external μ". See clause 5.3.4 for details.

NOTE 4: The external μ approach is also permitted for pure ML-DSA verification (NIST FIPS 204 [i.7], algorithm 8). In this case, the public verification key is needed for both the computation of the message representative and the rest of the signature verification process (see clause 5.4.3). However, separating the computation of the message representative from the rest of verification can be useful if the message itself is sensitive.

Finally, NIST FIPS 204 [i.7] also allows hedged or deterministic ML-DSA signature generation (see NIST FIPS 204 [i.7], section 3.4). In the hedged variant, each signature generation call includes a fresh 32-byte random value when deriving the ephemeral values from the message representative. In the deterministic variant, this 32-byte value is set to zero.

NOTE 5: Hedged ML-DSA and deterministic ML-DSA are considered variants of the same algorithm with the same algorithm identifier.

NOTE 6: Hedged ML-DSA is the default choice in NIST FIPS 204 [i.7].

5.3.2 Pure ML-DSA

The specification of pure ML-DSA signature generation in NIST FIPS 204 [i.7] (algorithm 2) samples a 32-byte value, formats the message to include the optional context string, and then generates a signature for the formatted message via an internal signature generation function using the 32-byte value as input (see Table 4).

NOTE 1: If no context string is provided, pure ML-DSA signature generation uses the empty string as the default.

NOTE 2: The deterministic variant of pure ML-DSA signature generation omits steps 4 to 7 and instead uses the fixed 32-byte value $rnd = 0x00 \ldots 00$ in step 9.

NOTE 3: When a private seed is being used in place of the full private signing key, the only change to pure ML-DSA signature generation is that the private seed needs to be expanded to recover the private key before the internal signature generation function is called in step 9.

Table 4: Pure ML-DSA hedged signature generation interface

ML-DSA.Sign(sk, M, ctx)		
Input:	Private signing key $\boldsymbol{sk}$, message $\boldsymbol{M}$ and (optional) context string $\boldsymbol{ctx}$	
Output:	Signature $\boldsymbol{\sigma}$, or Error	
1.	if len(ctx) > 255 then	
2.	return Error	(Context string too long)
3.	end if	
4.	$rnd \leftarrow$ RBG(32)	
5.	if rnd = NULL then	
6.	return Error	(RBG failure)
7.	end if	
8.	$M' \leftarrow 0x00 \,\|\|\, len(ctx) \,\|\|\, ctx \,\|\|\, M$	(Format message)
9.	$\sigma \leftarrow$ ML-DSA.Sign_internal(sk, M', rnd)	
10.	return σ	

5.3.3 HashML-DSA

The specification of HashML-DSA signature generation in NIST FIPS 204 [i.7] (algorithm 2) samples a 32-byte value, pre-hashes the message, formats it to include the optional context string and identifier for the pre-hash function, and then generates a signature for the formatted message via an internal signature generation function using the 32-byte value as input (see Table 5).

NOTE 1: If no context string is provided, HashML-DSA signature generation uses the empty string as the default.

NOTE 2: The deterministic variant of HashML-DSA signature generation omits steps 4 to 7 and instead uses the fixed 32-byte value $rnd = 0x00\ldots00$ in step 20.

NOTE 3: When a private seed is being used in place of the full private signing key, the only change to HashML-DSA signature generation is that the private seed needs to be expanded to recover the private key before the internal signature generation function is called in step 20.

Table 5: HashML-DSA hedged signature generation interface (fragment)

HashML-DSA.Sign($\boldsymbol{sk}$, $\boldsymbol{M}$, $\boldsymbol{ctx}$, $\boldsymbol{PH}$)	
Input:	Private signing key $\boldsymbol{sk}$, message $\boldsymbol{M}$, (optional) context string $\boldsymbol{ctx}$ and pre-hash function $\boldsymbol{PH}$
Output:	Signature $\boldsymbol{\sigma}$, or Error
1. if len(ctx) > 255 then	
2. return Error	(Context string too long)
3. end if	
4. $rnd \leftarrow$ RBG(32)	
5. if rnd = NULL then	
6. return Error	(RBG failure)
7. end if	
8. switch PH do	
9. case SHA-256:	
10. $OID \leftarrow 0x0609608648016503040201$	(SHA-256 algorithm identifier)
11. $PHM \leftarrow$ SHA256(M)	
⋮	
18. end switch	
19. $M' \leftarrow 0x01$ || len(ctx) || ctx || OID II PHM	(Format message)
20. $\sigma \leftarrow$ ML-DSA.Sign_internal(sk, M', rnd)	
21. return σ	

5.3.4 External μ

The message representative μ computed by the pure ML-DSA internal signature generation function (NIST FIPS 204 [i.7], algorithm 7) only depends on the formatted message and the hash of the public verification key. Further, the seed ρ'' used to derive the ephemeral private values in signing only depends on the message representative, the private signing key, and a 32-byte random value (see Table 6).

Consequently, pure ML-DSA signature generation can be cleanly separated into two parts:

- Computation of the message representative, which only requires access to the hash of the public verification key, the message and the optional context string (see Table 7); and
- Signing the message representative, which only requires access to the private signing key and the message representative (see Table 8).

As the first part does not involve the private signing key, this can be performed outside the cryptographic module containing the private key.

As the second part does not involve the original message, only the message representative needs to be passed to the cryptographic module to complete the signature generation.

NOTE 1: External μ is an approach to implementing pure ML-DSA. It uses the same pure ML-DSA algorithm identifiers. Key generation is the same as for pure ML-DSA and signatures produced by an external μ implementation can be verified by a non-external μ implementation.

NOTE 2: The computation of the message representative is the same for pure ML-DSA signature generation and verification.

NOTE 3: The external μ approach can be used with both the hedged and deterministic variants of pure ML-DSA.

Table 6: Pure ML-DSA internal signature generation function (fragment)

ML-DSA.Sign_internal($\boldsymbol{sk}$, $\boldsymbol{M'}$, $\boldsymbol{rnd}$)		
Input:	Private signing key $\boldsymbol{sk}$, formatted message $\boldsymbol{M'}$ and random value $\boldsymbol{rnd}$	
Output:	Signature $\boldsymbol{\sigma}$, or Error	
1.	$(\rho, K, tr, s_1, s_2, t_0) \leftarrow$ skDecode(sk)	
⋮		
6.	$\mu \leftarrow$ SHAKE256(tr || M', 512)	(Compute message representative)
7.	$\rho'' \leftarrow$ SHAKE256(K || rnd || μ, 512)	
⋮		
9.	$(z, h) \leftarrow$ NULL	
10.	while (z, h) = NULL do	
⋮		
23.	if $\|\|z\|\|_\infty \geq \gamma_1 - \beta$ or $\|\|r_0\|\|_\infty \geq \gamma_2 - \beta$ then $(z, h) \leftarrow$ NULL	(Rejection sampling checks)
24.	else	
⋮		
28.	if $\|\|ct_0\|\|_\infty \geq \gamma_2$ or wt(h) $> \omega$ then $(z, h) \leftarrow$ NULL	(Correctness checks)
29.	end if	
30.	end if	
⋮		
32.	end while	
⋮		
34.	return σ	

Table 7: External μ message representative computation interface

ExternalMu-ML-DSA.Prehash($\boldsymbol{pk}$, $\boldsymbol{M}$, $\boldsymbol{ctx}$)		
Input:	Public verification key $\boldsymbol{pk}$, message $\boldsymbol{M}$ and (optional) context string $\boldsymbol{ctx}$	
Output:	Message representative $\boldsymbol{\mu}$, or Error	
1.	if len(ctx) > 255 then	
2.	return Error	(Context string too long)
3.	end if	
4.	$M' \leftarrow 0x00$ || len(ctx) || ctx || M	(Format message)
5.	$tr \leftarrow$ SHAKE256(pk, 512)	
6.	$\mu \leftarrow$ SHAKE256(tr || M', 512)	(Compute message representative)
7.	return μ	

Table 8: External μ hedged signature generation interface (fragment)

ExternalMu-ML-DSA.Sign(sk, μ)		
Input:	Private signing key $\boldsymbol{sk}$ and message representative $\boldsymbol{\mu}$	
Output:	Signature $\boldsymbol{\sigma}$, or Error	
1.	$rnd \leftarrow$ RBG(32)	
2.	if rnd = NULL then	
3.	return Error	(RBG failure)
4.	end if	
5.	(ρ, K, tr, s_1, s_2, t_0) $\leftarrow$ skDecode(sk)	
⋮		
10.	$\rho'' \leftarrow$ SHAKE256(K || rnd || μ, 512)	
⋮		
37.	return σ	

5.4 Signature verification

5.4.1 Pure ML-DSA

The specification of pure ML-DSA signature verification in NIST FIPS 204 [i.7] (algorithm 3) simply formats the message to include the optional context string and then calls an internal signature verification function (see Table 9).

NOTE 1: If no context string is provided, pure ML-DSA signature verification uses the empty string as the default.

NOTE 2: If the context string is longer than 255 bytes, pure ML-DSA signature verification returns an error rather than rejecting the signature.

Table 9: Pure ML-DSA signature verification interface

ML-DSA.Verify($\boldsymbol{pk}$, $\boldsymbol{M}$, $\boldsymbol{\sigma}$, $\boldsymbol{ctx}$)		
Input:	Public verification key $\boldsymbol{pk}$, message $\boldsymbol{M}$, signature $\boldsymbol{\sigma}$ and (optional) context string $\boldsymbol{ctx}$	
Output:	Boolean, or Error	
1.	if len(ctx) > 255 then	
2.	return Error	(Context string too long)
3.	end if	
4.	$M' \leftarrow 0x00$ || $len(ctx)$ || ctx || M	(Format message)
5.	return ML-DSA.Verify_internal(pk, M', σ)	

5.4.2 HashML-DSA

The specification of HashML-DSA signature verification in NIST FIPS 204 [i.7] (algorithm 5) simply pre-hashes the message, formats it to include the optional context string and identifier for the pre-hash function, and then calls an internal signature verification function (see Table 10).

NOTE 1: If no context string is provided, HashML-DSA signature verification uses the empty string as the default.

NOTE 2: If the context string is longer than 255 bytes, HashML-DSA signature verification rejects the signature.

Table 10: HashML-DSA signature verification interface (fragment)

HashML-DSA.Verify(pk, M, σ, ctx, PH)	
Input:	Public verification key pk, message M, signature σ, (optional) context string ctx and pre-hash function PH
Output:	Boolean
1.	if len(ctx) > 255 then
2.	return False (Context string too long)
3.	end if
4.	switch PH do
5.	case SHA-256:
6.	$OID \leftarrow 0x0609608648016503040201$ (SHA-256 algorithm identifier)
7.	$PHM \leftarrow$ SHA256(M)
⋮	
14.	end switch
15.	$M' \leftarrow 0x01$ || len(ctx) || ctx || OID || PHM (Format message)
16.	return ML-DSA.Verify_internal(pk, M', σ)

5.4.3 External μ

The message representative μ computed by the pure ML-DSA internal verification function (NIST FIPS 204 [i.7], algorithm 8) only depends on the formatted message and the hash of the public verification key. The challenge value $\tilde{c}$ checked during verification only depends on the message representative and the signer's commitment w_1' which is reconstructed from the public key and signature (see Table 11).

Consequently, pure ML-DSA signature verification can be cleanly separated into two parts:

- Computation of the message representative, which requires access to the hash of the public verification key, the message and the optional context string (see Table 7); and
- Verifying the signed message representative, which requires access to the public verification key, the message representative and the signature (see Table 12).

Both parts involve the public verification key, but they can be performed in separate cryptographic modules. As the second part does not involve the original message, only the message representative needs to be passed between the two modules [i.8].

NOTE 1: External μ is an approach to implementing pure ML-DSA. It uses the same pure ML-DSA algorithm identifiers. Key generation is the same as for pure ML-DSA and signatures produced by a non-external μ implementation can be verified by an external μ implementation.

NOTE 2: The computation of the message representative is the same for pure ML-DSA signature generation and verification.

Table 11: Pure ML-DSA internal signature function (fragment)

ML-DSA.Verify_internal(pk, M', σ)	
Input:	Public verification key pk, formatted message M' and signature σ
Output:	Boolean
1.	$(\rho, t_1) \leftarrow$ pkDecode(pk)
2.	$(\tilde{c}, z, h) \leftarrow$ sigDecode(σ)
⋮	
6.	$tr \leftarrow$ SHAKE256(pk, 512)
7.	$\mu \leftarrow$ SHAKE256(tr || M', 512) (Compute message representative)
8.	$c \leftarrow$ SampleInBall($\tilde{c}$)
⋮	
12.	$\tilde{c}' \leftarrow$ SHAKE256(μ || w_1', 2λ) (Recompute challenge value)
13.	return ($\|z\|_\infty < \gamma_1 - \beta$) and ($\tilde{c} = \tilde{c}'$)

Table 12: External μ signature verification interface (fragment)

ExternalMu-ML-DSA.Verify(pk, μ, σ)	
Input:	Public verification key $\boldsymbol{pk}$, message representative $\boldsymbol{\mu}$ and signature $\boldsymbol{\sigma}$
Output:	Boolean
1.	$(\rho, t_1) \leftarrow$ pkDecode(pk)
2.	$(\tilde{c}, z, h) \leftarrow$ sigDecode(σ)
⋮	
6.	$c \leftarrow$ SampleInBall($\tilde{c}$)
⋮	
10.	$\tilde{c}' \leftarrow$ SHAKE256($\mu \parallel w_1'$, 2λ) (Recompute challenge value)
11.	return $(\|z\|_\infty < \gamma_1 - \beta)$ and $(\tilde{c} = \tilde{c}')$

6 General implementation considerations

6.1 Perform input validation

6.1.1 Signature generation inputs

NIST FIPS 204 [i.7] does not require any checks on the private signing key before it can be used in pure ML-DSA or HashML-DSA signature generation. This is not necessarily an issue if the validity of the private key can be assured in other ways.

EXAMPLE 1: A private signing key generated by the implementation and subsequently stored in a way that prevents modification does not need to be validated.

EXAMPLE 2: A private signing key that has been provided by a trusted third party with assurances that it is valid and subsequently stored in a way that prevents modification does not need to be validated.

Decoding a malformed private signing key can return values that are not in the correct range for a valid private key and could cause signature generation to behave unpredictably. NIST FIPS 204 [i.7] recommends that ML-DSA implementations do not use private keys that have not been self-generated or provided by a trusted third party (see NIST FIPS 204 [i.7], section 7.2).

EXAMPLE 3: If the coefficients of the private signing key values s_1 or s_2 are larger than expected, this could invalidate the rejection sampling checks in ML-DSA.Sign_internal and cause signatures to leak information about the private key (see NIST FIPS 204 [i.7], section 3.3).

NOTE 1: Although not required by NIST FIPS 204, it would be straightforward to check that the coefficients of s_1 and s_2 are in the expected range during private key decoding in skDecode (NIST FIPS 204 [i.7], algorithm 25).

NOTE 2: This check does not guarantee that the private key is valid output from ML-DSA key generation. Full validation of an ML-DSA private key is not possible without the private seed used to generate it.

When a private seed is being used in place of the private signing key, the only check possible is that it is a byte string of length 32 bytes.

6.1.2 Signature verification inputs

NIST FIPS 204 [i.7] requires that implementations check the public verification key and signature are byte strings of the expected lengths before they can be used in pure ML-DSA or HashML-DSA signature verification. If either check fails, the implementation is required to reject the signature (see NIST FIPS 204 [i.7], section 3.6.2).

EXAMPLE 1: An implementation that ignored the extra bytes in a longer-than-expected signature instead of rejecting it would potentially allow an adversary to modify a valid signature for a message to give a different signature for the same message that would also be accepted by the implementation. This violates the strong unforgeability of ML-DSA (see clause B.1).

EXAMPLE 2: An implementation that attempted to process a shorter-than-expected signature instead of rejecting it could potentially leak information from memory.

The ML-DSA signature decoding process includes checks that the hint field h contained in the signature has been encoded correctly (see HintBitUnpack in NIST FIPS 204 [i.7], algorithm 21). These checks ensure that there is a single valid encoding for the hint field. Any signature that fails the checks is rejected.

EXAMPLE 3: An implementation that omits some or all of the hint field encoding checks in HintBitUnpack would allow an adversary to modify a valid signature for a message to give a different signature for the same message that would also be accepted by the implementation (see, for example, CVE-2026-24850). This violates the strong unforgeability of ML-DSA (see clause B.1).

NOTE 1: When implemented correctly, any signature that passes ML-DSA verification is guaranteed to be valid output from ML-DSA signature generation due to the unforgeability of ML-DSA (see clause B.1).

The initial public draft of NIST SP 800-57A Rev. 6 [i.9] (section 4.4.3) requires verifiers to obtain assurance that the public verification key is valid before attempting to verify a signature. The methods for obtaining assurance specified in NIST SP 800-89 [i.10] (section 5.2) include generation of the key pair by a trusted third party, or through explicit validation of the public key by the verifier or a trusted third party.

NOTE 2: NIST SP 800-89 [i.10] has not been updated since the publication of NIST FIPS 204 [i.7]. The only explicit public key validation processes described are for traditional signature algorithms.

NOTE 3: Length and type checks ensure that the public verification key will decode to a 32-byte seed and a vector of polynomials whose coefficients are in the expected range (see NIST FIPS 204 [i.7], algorithm 23). This does not guarantee that the public key is valid output from ML-DSA key generation. Full validation of an ML-DSA public key is not possible without the private seed used to generate it.

6.1.3 HashML-DSA inputs

NIST FIPS 204 [i.7] (section 5.4) requires that the function used to pre-hash the message in HashML-DSA provides at least the same level of security as the ML-DSA parameter set. If an implementation allows the pre-hash function to be given as an input to HashML-DSA.Sign (NIST FIPS 204 [i.7], algorithm 4) or HashML-DSA.Verify (NIST FIPS 204 [i.7], algorithm 5), then it is important to check that it is appropriate for the ML-DSA parameter set being used.

EXAMPLE: Using SHA-256 as the pre-hash function with the ML-DSA-87 parameter set would reduce the overall security provided by HashML-DSA (see NIST SP 800-57A Rev. 6 [i.9], section 4.6.2).

NOTE: HashML-DSA.Sign includes an identifier for the pre-hash function in the formatted message signed by the private key (see Table 5). If an adversary attempted to use a different pre-hash function in HashML-DSA.Verify, the signature would be rejected (unlike, for example, CVE-2026-5194).

6.1.4 External μ inputs

NIST FIPS 204 [i.7] allows the message representative μ to be computed separately from the rest of the ML-DSA signing and verification processes (see clauses 5.3.4 and 5.4.3). Preparation of the message representative involves the public verification key, message and optional context string. Although not required by NIST FIPS 204, it is good practice to check the length of the public key before computing the message representative.

NOTE: If the message representative is being computed outside the cryptographic module used to manage the private signing key, it is important to ensure that the correct public verification key is being used. This could be achieved by passing the hash of the public key to the module alongside the message representative and checking that this matches the hash value contained in the module's private key.

6.1.5 Internal algorithms

NIST FIPS 204 [i.7] does not require input validation for any of the internal ML-DSA algorithms. However, it is good practice for an implementation to perform input validation on all functions, especially if these functions are exposed to the user, for example for testing purposes.

EXAMPLE: ML-DSA.KeyGen_internal (NIST FIPS 204 [i.7], algorithm 6) deterministically generates a key pair from a 32-byte seed. Allowing the function to use a shorter-than-expected byte string could potentially lead to predictable key pairs.

NOTE: General recommendations for input validation can be found in clause 6.2 of ETSI TR 104 239-1 [i.2].

6.2 Perform output validation

6.2.1 Rejection sampling checks

ML-DSA uses the Fiat-Shamir approach [i.32] to convert an interactive proof-of-knowledge protocol into a signature algorithm. However, as with other lattice-based signatures [i.40], a straightforward application of the Fiat-Shamir approach would lead to signatures that leak information about the private signing key. Consequently, ML-DSA uses rejection sampling during signature generation to ensure that the signature distribution is independent of the private key [i.38].

Specifically, ML-DSA.Sign_internal includes checks on the sizes of the coefficients in z and r_0 (see Table 6, step 23). If either check fails, the signature is rejected and signature generation restarts with a new ephemeral private value.

EXAMPLE: An implementation that omitted the coefficient size checks or used incorrect bounds would potentially output signatures that leak information about the private signing key.

NOTE 1: ML-DSA.Verify_internal includes the same check on the size of the coefficients in z. Any signature that fails this check will be rejected.

ML-DSA.Sign_internal also includes checks on the size of the coefficients in ct_0 and the number of 1's in the hint field h (see Table 6, step 28). These checks ensure that the signature will verify correctly (see [i.14], section 3.3) and that the hint field will be encoded correctly. If either check fails, the signature is rejected and signature generation restarts with a new ephemeral private value.

NOTE 2: ML-DSA.Verify_internal includes a check on the encoding of the hint field (see clause 6.1.2). Any signature that fails this check will be rejected.

Verifying an ML-DSA signature after it has been generated, as is suggested for high-value messages in NIST FIPS 186-5 [i.6], will detect when some of the signature generation checks have been omitted, but it cannot detect all of them.

6.2.2 Pairwise consistency test

NIST's additional implementation guidance for FIPS 140-3 [i.13] (section 10.3.A) requires that cryptographic modules perform a pairwise consistency test before a module-generated ML-DSA key can be exported or used for the first time. Although NIST FIPS 204 [i.7] does not describe a pairwise consistency test, following ISO/IEC 24759 [i.5] (VE10.35.02) this is achieved by generating and verifying a signature.

EXAMPLE: The pairwise consistency test for an ML-DSA key pair could include calling ML-DSA.Sign with the private signing key and a fixed message $m = 0x00 \ldots 00$, then checking that the signature output by the function is accepted by ML-DSA.Verify with the corresponding public verification key. If the signature is rejected, the pairwise consistency test fails.

NOTE 1: As the pairwise consistency test involves ML-DSA signature verification, it is important to validate the public verification key before performing this check (see clause 6.1.2).

NOTE 2: The implementation guidance only requires that a cryptographic module performs the pairwise consistency test for ML-DSA key pairs generated by that module. It is not required for imported key pairs that were generated outside the module.

NOTE 3: This test does not guarantee that the key pair is valid output from ML-DSA key generation. Full validation of the key pair is not possible without access to the private seed used to generate it.

NOTE 4: NIST FIPS 204 [i.7] does not require any form of key pair validation.

6.2.3 Self testing

For high-assurance applications, it is often recommended that implementations perform a self-test before an algorithm is used for the first time after start-up. Self-tests typically involve checking that the algorithm returns the expected output when provided known inputs. They are intended to detect transient failures caused by hardware faults or implementation corruption, including tampering by an adversary.

EXAMPLE 1: A self-test for ML-DSA key generation could include calling ML-DSA.KeyGen_internal with the test seed $\xi = 0x00 \ldots 00$, then checking that the algorithm returns the expected public verification key and private signing key.

EXAMPLE 2: A self-test for ML-DSA signature generation could include calling ML-DSA.Sign_internal with a test private signing key sk and the inputs $M' = 0x00 \ldots 00$ and $rnd = 0x00 \ldots 00$, then checking that the algorithm returns the expected signature.

NOTE 1: Self-tests are distinct from the implementation testing described in clause 8.1.

NIST's implementation guidance for FIPS 140-3 [i.13] (section 10.3.A) requires that cryptographic modules include self-tests for ML-DSA signature generation, signature verification and key generation when the relevant algorithms are implemented.

NOTE 2: The implementation guidance requires that the signature generation self-tests include cases that are expected to fail the different rejection sampling checks in ML-DSA.Sign_internal (NIST FIPS 204 [i.7], algorithm 7).

NOTE 3: The implementation guidance only requires a signature verification self-test where the signature is valid, but it is good practice to include at least one additional case where the signature is expected to be rejected.

NOTE 4: The implementation guidance only requires that the key generation self-test checks that the algorithm returns the expected private signing key as this includes the hash of the public verification key.

NOTE 5: If an implementation supports multiple ML-DSA parameter sets, then the implementation guidance only requires that self-tests are performed for one of the parameter sets.

NOTE 6: If an implementation supports both the pure ML-DSA and HashML-DSA variants using the same set of internal functions, then the implementation guidance only requires that self-tests are performed using one of the two variants.

6.2.4 Public key linting

More detailed public verification key and certificate checks have been proposed in [i.36]. In addition to the public key length and format checks from clause 6.1.2, these include checks on the length and distribution of the seed ρ contained in the public key and checks on the key usage extensions and algorithm identifiers when the public key is provided in a certificate.

6.3 Prevent leakage of intermediate values

6.3.1 Zeroise intermediate values after use

It is generally recommended that implementations zeroise intermediate values before the key generation or signature generation algorithms return the output.

EXAMPLE 1: Recovering the input ρ' to ExpandS in ML-DSA.KeyGen_internal (NIST FIPS 204 [i.7], algorithm 6) would allow an adversary to recover the private key value s_1 directly.

EXAMPLE 2: Recovering the input ρ'' to ExpandMask in ML-DSA.Sign_internal (NIST FIPS 204 [i.7], algorithm 7) would allow an adversary to recover the private key value s_1 key from the corresponding signature with reasonable probability.

NOTE 1: Recovering s_1 is sufficient for an adversary to forge signatures for arbitrary messages [i.44].

Intermediate values computed during verification depend only on the message, signature and public verification key. However, these can still be sensitive if, for example, the message is confidential.

NIST FIPS 204 [i.7] requires that any potentially sensitive intermediate values are zeroised with two exceptions:

- The private seed ξ used in key generation (see clause 5.2.3); and
- The matrix $\hat{A}$ derived from the public seed ρ via the ExpandA function ([i.7], algorithm 32).

NOTE 2: The private seed ξ should be stored with the same level of protection as the private signing key.

NOTE 3: The matrix $\hat{A}$ is computed during signature generation and verification. Storing $\hat{A}$ between signature generation or verification calls can be more efficient that re-computing it each time. It should be stored with the same level of protection as the public verification key.

6.3.2 Limit external access to internal functions

Access to the internal functions ML-DSA.KeyGen_internal, ML-DSA.Sign_internal and ML-DSA.Verify_internal can be necessary for testing purposes. However, NIST FIPS 204 [i.7] (section 6) recommends that their interfaces are not made available to applications for any purpose other than testing.

EXAMPLE 1: Direct access to ML-DSA.KeyGen_internal would bypass secure random bit generation (see clause 6.5).

EXAMPLE 2: Direct access to ML-DSA.Verify_internal would bypass public key and signature length checks (see clause 6.1.2).

EXAMPLE 3: Direct access to ExternalMu-ML-DSA.Sign would potentially bypass checks on the choice of pre-hash function (see clause 6.1.3).

EXAMPLE 4: Direct access to ExternalMu-ML-DSA.Verify would potentially remove the binding between the signature and the public verification key (see clause B.2).

6.4 Handle errors gracefully

6.4.1 Specified errors

NIST FIPS 204 [i.7] requires that ML-DSA signature generation and verification return an explicit error when they are passed a context string longer than 255 bytes.

ML-DSA key generation returns an explicit error when a call to the random bit generator fails. If an implementation does not check that the call to the random bit generator was successful, or if it ignores any failures, this can result in predictable key pairs.

Hedged ML-DSA signature generation also returns an explicit error when a call to the random bit generator fails. In this case, ignoring a failure is less of a direct security threat (see clause 6.5) but it could still cause signature generation to behave unpredictably.

A pairwise consistency test (see clause 6.2.2) can return an error caused by a failure of the test itself or, if it uses random messages, a failure of the random bit generator.

6.4.2 Unspecified errors

ML-DSA.Sign_internal (NIST FIPS 204 [i.7], algorithm 7) uses a rejection sampling loop that can take a variable number of iterations to produce a valid signature.

Similarly, the supporting functions SampleInBall (NIST FIPS 204 [i.7], algorithm 29), RejNTTPoly (NIST FIPS 204 [i.7], algorithm 30), and RejBoundedPoly (NIST FIPS 204 [i.7], algorithm 31) all use rejection sampling to produce polynomials with the desired properties.

- RejBoundedPoly is used during key generation;

- SampleInBall is used during signature generation and verification; and
- RejNTTPoly is used during key generation, signature generation and verification.

Although NIST FIPS 204 recommends that implementations do not limit the number of iterations, annex C of NIST FIPS 204 [i.7] gives lower bounds for which the different functions will succeed with overwhelming probability. If there is a fault or a mistake in the implementation, then the number of iterations could exceed this bound. In that case, a bounded-loop implementation would return an error that needs to be checked and handled by the calling function.

Other explicit errors can occur depending on any additional validation or consistency checks included in an ML-DSA implementation; for example, checking that a memory allocation has succeeded.

NOTE: General recommendations for error handling can be found in clause 6.5 of ETSI TR 104 239-1 [i.2]. In particular, it is important that implementations that indicate errors through a status value do not also return the output that might have caused the error since this could be inadvertently used elsewhere.

6.4.3 Verification failures

NIST FIPS 204 [i.7] (section 3.6.2) requires that the lengths of the public verification key and signature are checked before they can be used in ML-DSA verification (see clause 6.1.2). However, a failure will not produce an explicit error. Instead, an implementation is required to reject the signature as invalid; that is, ML-DSA verification returns false.

6.5 Randomness

ML-DSA key generation needs a source of random bits. Weakly generated random bits can lead to predictable key pairs. Consequently, NIST FIPS 204 [i.7] (section 3.6.1) requires that implementations use an approved random bit generator for key generation with an appropriate level of security of the parameter set.

Hedged ML-DSA additionally needs a source of random bits at the time of signature generation. Weakly generated random bits do not lead to weak signatures or threaten the security of the private key directly. In this case, NIST FIPS 204 [i.7] (section 3.6.1) only recommends that implementations use an approved random bit generator but does not require it. Hedged signing with a weak random bit generator can still be preferable over deterministic signing for side-channel or fault attack reasons (see clause 7.2).

NOTE: Deterministic ML-DSA signing uses the fixed 32-byte value $rnd = 0x00 \ldots 00$.

Although NIST FIPS 204 [i.7] (algorithm 7) allows the message representative μ to be computed outside of the cryptographic module that performs signature generation, it requires that the 32-byte random value used in hedged signing is generated by the module (see NIST FIPS 204 [i.7], section 5.4).

7 Side-channel and fault attack considerations

7.1 Key Generation

7.1.1 Overview

As discussed in clause 5.2, ML-DSA.KeyGen generates a single random 32-byte seed and then deterministically expands this to produce a public-private key pair. This implies that the relevant side-channel attacks are single trace attacks.

At first glance, the key generation algorithm is called only once per public-private key pair, which implies a limited scope for side-channel or fault attacks. However, NIST FIPS 204 [i.7] permits the private key to be stored as either a seed or in expanded format (see clause 5.2.1). Before use in signature generation, a key stored in seed format should be expanded out into the full private key. This requires a call to the internal function ML-DSA.KeyGen_internal.

If a device is storing the key in seed format, an adversary might be able to repeatedly trigger the key expansion routine by inducing signing requests. This could be used to reduce the noise in a single trace attack by aggregating many traces, and may have more serious consequences for fault attacks.

If side-channels or fault attacks are considered a high risk in a device's threat model, then it may be preferrable to store ML-DSA private keys in expanded format, assuming that the protected memory requirements can be met (see clause 5.2.3).

7.1.2 Timing Analysis Considerations

As discussed in the general guidance, ETSI TR 104 239-1 [i.2], it is necessary to avoid secret-dependent branching, variable time operations and secret-dependent memory addressing in cryptographic code. The pseudo-code published in NIST FIPS 204 [i.7] for ML-DSA.KeyGen, ML-DSA.KeyGen_internal and their subroutines does not contain secret-dependent branching or secret-dependent memory addressing. Rejection sampling is employed in the expansion of the private key values $s_1$and s_2, and can lead to a variation in the runtime for different seed values. These variations will be largest when an additional call to the SHAKE256 generator is required. This can be mitigated by oversampling to produce sufficient random in the vast majority of cases, at the cost of a small number of additional calls to SHAKE256 per key generation.

Fundamental operations will need to be assessed on a per-architecture basis. Division and modular reduction operations are particularly likely to be non-constant time.

EXAMPLE 1: A secret-dependent timing variation introduced by the division operator occurred in many early implementations of the post-quantum lattice-based scheme Kyber, the post-quantum scheme on which ML-KEM is based [i.19].

Where constant-time fundamental operations do not exist, implementations could consider replacing them with functionally equivalent algorithms designed to be constant-time. Commonly used examples for modular reduction and multiplication are Barrett reduction and Montgomery multiplication respectively. The precise functional equivalence can be targeted to known properties of data at particular points in the algorithm to allow for optimizations.

EXAMPLE 2: Appendix A of NIST FIPS 204 [i.7] describes Montgomery multiplication, which can be used to calculate $a \cdot b \pmod q$ using only reduction and division by 2^{32}, which can be done through straightforward bit manipulations.

7.1.3 Power Analysis Considerations

As discussed in clause 7.1.1, the deterministic nature of the ML-DSA.KeyGen_internal implies that when power analysis attacks are part of a device's threat model, only simple power analysis attacks apply. Simple power analysis typically recovers the Hamming weight of secret values. The entropy loss depends on the distribution of values.

EXAMPLE 1: An AES key byte distributed uniformly in the range 0-255 has an entropy of 8 bits. Learning its Hamming weight leads to an average Shannon entropy loss of 2,5 bits [i.33].

EXAMPLE 2: Coefficients of the ML-DSA secret polynomials s_1 and s_2 are uniformly distributed in the range $\pm\eta$ where $\eta \in \{2, 4\}$. If $\eta = 2$, each coefficient provides 2,32 bits of entropy. The representation of negative values is not specified in NIST FIPS 204 [i.7], but they could be represented as unsigned integers $\bmod q$, as in ML-KEM. In this case, there is a nearly unique Hamming weight per value. Learning the Hamming weight leads to an average Shannon entropy loss of 1,92 bits.

The above example demonstrates that in worst case, where an adversary can perfectly learn the Hamming weights of an ML-DSA secret polynomial, a significant security loss occurs.

For efficiency reasons, s_1 is transformed into the NTT domain before multiplication with the public matrix $\hat{A}$. In the NTT, the coefficients are more uniformly distributed, and so learning Hamming weights of values is less useful to an adversary. Each coefficient in the NTT representation depends on many coefficients in the polynomial representation and vice versa. This makes it harder for an adversary to guess a small part of the secret in the polynomial representation and confirm their guess by correlating with later values being processed in the NTT representation. However, the NTT calculation and its inverse can be used for this guess-and-check attack [i.45].

The secret polynomials are stored in the polynomial domain using the skEncode function (NIST FIPS 204 [i.7], algorithm 24). Again, the limited range of coefficients makes the values more susceptible to Simple Power Analysis when processed and stored in this fashion.

A commonly employed mitigation for defending against simple power analysis attacks is applying random permutation to loops. This randomization is applied on a per-call basis, and prevents an adversary from correlating learned Hamming weights with the correct coefficient index. The permutations should be considered secret values, but, since they are refreshed on a per-call basis, this is unlikely to introduce exploitable secret-dependent memory addressing (see clause 7.1.2).

EXAMPLE 3: In the ML-DSA.KeyGen routine in NIST FIPS 204 [i.7], loop randomization could be employed within ExpandS (NIST FIPS 204 [i.7], algorithm 33), the NTT (NIST FIPS 204 [i.7], algorithm 41) and NTT^{-1} ([i.7], algorithm 42) functions and the BitPack (NIST FIPS 204 [i.7], algorithm 1) function within skEncode (NIST FIPS 204 [i.7], algorithm 24).

EXAMPLE 4: The matrix multiplication calculation of $\hat{A} \circ \widehat{s_1}$ provides an adversary with several observations of each secret coefficient being multiplied with public values. This could be viewed as providing several traces in a differential power analysis attack. Randomizing the loop order would prevent an adversary from labelling each "trace" with the correct public value to carry out guess-and-check correlations with small parts of the secret value.

7.1.4 Fault Attack Considerations

An adversary attempting to fault the ML-DSA.KeyGen process will be trying to learn or modify the value of the long-term private key. This could be done by faulting the random generation or expansion in ML-DSA.KeyGen (NIST FIPS 204 [i.7], algorithm 1) and ML-DSA.KeyGen_internal (NIST FIPS 204 [i.7], algorithm 6). As discussed in clause 6.5, NIST FIPS 204 [i.7] (section 3.6.1) requires that implementations use an approved random bit generator. Where fault attacks are part of the threat model, this random bit generator will require additional protections that are out of scope of the present document. ML-DSA.KeyGen already includes an explicit check that the random returned value is not null. Faulting the random expansion in ML-DSA.KeyGen_internal will only be useful if the key is stored in semi-expanded format, or if the adversary can consistently fault the key expansion process.

If an ML-DSA key is stored in the seed format, an adversary may be able to repeatedly trigger the key expansion process as part of signing. In this scenario, faulting the expansion may leak information about the private key. The information leaked will depend on the output available to the adversary: it is assumed that they do not see the expanded private key, but they may see the public key or a signature produced from the faulted key.

EXAMPLE 1: An adversary sees the public key returned from the key expansion process and signatures produced from that public key. They fault the addition in step 5 of ML-DSA.KeyGen_internal (NIST FIPS 204 [i.7], algorithm 6), so that the public key returned is the high bits of $t' = A.\,s_1$. They recover the low bits from a few signatures using this value, allowing them to recover s_1 and then s_2.

EXAMPLE 2: An adversary can observe only signatures returned. They can skip an instruction in the skEncode subroutine (NIST FIPS 204 [i.7], algorithm 24), ensuring one coefficient of the private key is set to zero. By checking the validity of signatures produced with this faulted private key, they learn if the true value of the coefficient is zero.

Each of these examples are illustrative only: enumerating all possible faults and designing individual protections is extremely expensive. As discussed in the general guidance, ETSI TR 104 239-1 [i.2], a simpler mitigation is to repeat steps at risk of faulting and compare the outputs, only proceeding if output agrees. This requires an adversary to induce a precise pattern of consistent faults per iteration, which is generally considered extremely difficult.

Reducing an adversary's ability to target a particular instruction by introducing random delays or loop shuffling can limit the effectiveness of fault attacks. Tampering controls can provide an additional level of assurance.

7.2 Signature generation

7.2.1 Overview

An adversary attacking the signing algorithm may be trying to learn information about the long-term private key or to produce signatures for messages they control. In some cases, the message itself may be sensitive.

If the message is considered sensitive, NIST FIPS 204 [i.7] allows for the message representative to be calculated elsewhere and passed into the ML-DSA.Sign_internal function - the "external μ" version of ML-DSA discussed in clause 6.1.4. The message representative μ is produced via a SHAKE256 hash of the message together with some information about the public key, i.e. μ is a salted hash of the message. Therefore, μ should not be sensitive even if the message itself is sensitive.

It should be noted that the public value $\tilde{c}$ is a SHAKE256 hash of μ and a value computable from the signature. Therefore, if the message M is sufficiently low entropy to be exhaustible, it can be recovered by a "password guessing" attack on $\tilde{c}$. Hence, protocol security should not rely on a low-entropy message remaining secret.

ML-DSA.Sign has two modes: hedged and deterministic (see clause 5.3.1). The hedged mode incorporates a fresh, secret random value into the pseudorandom generation for each call to the signing function, whereas the deterministic version will produce the same signature for a given set of input values. It is very important that the value *y* is not re-used for different messages. The hedged version aims to prevent fault-induced re-use.

7.2.2 Timing Analysis Considerations

As discussed in the general guidance, ETSI TR 104 239-1 [i.2], it is necessary to avoid secret-dependent branching, variable time operations and secret-dependent memory addressing in cryptographic code. The pseudo-code published in NIST FIPS 204 [i.7] for ML-DSA.Sign, ML-DSA.Sign_internal and their subroutines does not contain secret-dependent memory addressing.

The rejection sampling loop will take a variable number of iterations. For the deterministic version of ML-DSA, this is entirely determined by the randomness derived from the message. If the message is sensitive, then the number of loops will reveal some information about the message. Therefore, the hedged version should be used if the message being signed is sensitive. A constant time implementation of SHAKE256 should also be used to calculate the message representative.

As discussed in clause 7.1.2, fundamental operations will need to be assessed on a per-architecture basis. Division and modular reduction operations are particularly likely to be non-constant time.

7.2.3 Power Analysis Considerations

As discussed in clause 7.1.3, the limited number of Hamming weight values taken by the coefficients of the secret polynomials s_1 and s_2 are a concern for simple power analysis attacks, where an adversary tries to learn secret values by observing their Hamming weights.

EXAMPLE 1: Large amounts of information about s_1, s_2 can be recovered from the BitUnpack routine (NIST FIPS 204 [i.7], algorithm 19) used in skDecode [i.51].

EXAMPLE 2: When operating on the secret polynomials, the NTT function ([i.7], algorithm 41) and its inverse NTT^{-1} (NIST FIPS 204 [i.7], algorithm 42) can be vulnerable to simple power analysis [i.45].

As suggested in clause 7.1.3, loop shuffling can be employed to mitigate simple power analysis attacks. If the message is considered sensitive, a protected implementation of the SHAKE256 hash function will also be required during the calculation of the message representation.

Differential Power Analysis (DPA) and Correlational Power Analysis (CPA) apply when a secret value interacts with a changing public value. These risks apply whether or not the adversary can choose or manipulate the public values.

EXAMPLE 3: The public value of $\tilde{c}$ changes on a per-message basis and on a per-signature basis for hedged mode. It determines the values of c and $\hat{c}$, which interact with the secret values s_1 and s_2 in both the polynomial and NTT domain in ML-DSA.Sign_internal (NIST FIPS 204 [i.7], algorithm 7). These calculations should therefore all be protected from DPA and CPA attacks.

Implementations needing to mitigate power analysis risks will require protected implementations of many of the subroutines in ML-DSA.Sign_internal [i.24], [i.25], typically using masking to decouple power draw from sensitive internal states. This will incur significant overheads in the running time and randomness requirements of the algorithm. Careful evaluation of proposed countermeasures will be required, including experimental measurement that leakage has been mitigated on the devices where the protected implementation is expected to run.

7.2.4 Fault Attack Considerations

An adversary faulting the ML-DSA signing process may be attempting to learn information about the secret signing key or modify the message being signed into something of their choosing which the device would not usually sign. In some cases, the message itself may be sensitive.

EXAMPLE 1: An implementation of ML-DSA is using the deterministic variant. The adversary flips a bit in the message representative μ after the seed ρ'' is calculated but before the challenge hash $\tilde{c}$ is calculated. This gives them two signatures with the same value of y, breaking the security of the scheme. The hedged variant of ML-DSA will provide more protection against this attack.

EXAMPLE 2: An adversary is allowed to request signatures of messages with a particular Boolean value set to '0'. The device checks a given message satisfies this property outside of the cryptographic module and calls the ML-DSA.Sign function. The adversary flips the necessary bit before the message representative is calculated and obtains a signature for a message with the Boolean value set to '1'.

Each of these examples are illustrative only: enumerating all possible faults and designing individual protections is extremely expensive. As discussed in the general guidance, ETSI TR 104 239-1 [i.2], a simpler mitigation is to repeat steps at risk of faulting and compare the outputs, only proceeding if output agrees. This requires an adversary to accurately induce more than one fault per iteration, which is generally considered extremely difficult.

Reducing an adversary's ability to target a particular instruction by introducing random delays or loop shuffling can limit the effectiveness of fault attacks. Tampering controls can provide an additional level of assurance.

7.3 Signature verification

7.3.1 Overview

The verification procedure takes an ML-DSA public key, a message and a signature. Unless the message itself is considered sensitive, there is no secret information processed by this routine. This means that side-channel analysis is likely not a concern. However, fault attacks may be of significant concern, due to the risks of an adversary being able to supply a false signature and have it successfully verified.

If the message is considered sensitive, NIST FIPS 204 [i.7] allows for the message representative to be calculated elsewhere and passed into the ML-DSA.Verify_internal function - the "external μ" version of ML-DSA. The message representative μ is produced via a SHAKE256 hash of the message together with some information about the public key, i.e. μ is a salted hash of the message. Therefore, μ should not be sensitive even if the message itself is sensitive.

It should be noted that the public value $\tilde{c}$ is a SHAKE256 hash of μ and a value computable from the signature. Therefore, if M is sufficiently low entropy to be exhaustible, it can be recovered by a "password guessing" attack on $\tilde{c}$. Hence, protocol security should not rely on a low-entropy message remaining secret.

7.3.2 Timing Analysis Considerations

In most cases, no secret information is processed by the verification routine, so timing analysis is not a concern. If the message is considered sensitive, the message representative μ should be calculated using a constant time implementation of SHAKE256.

7.3.3 Power Analysis Considerations

In most cases, no secret information is processed by the verification routine, so power analysis is not a concern. If the message is considered sensitive, the message representative μ should be calculated using a protected implementation of SHAKE256.

7.3.4 Fault Attack Considerations

An adversary attacking the verification procedure in ML-DSA.Verify will be attempting to force the procedure to return true, verifying a message of their choosing.

EXAMPLE 1: The comparison function on step 13 of ML-DSA.Verify_internal (NIST FIPS 204 [i.7], algorithm 8) is an obvious target, and a fault-protected implementation should be used.

EXAMPLE 2: An adversary possesses a valid signature for a given message. They flip one bit in the message and pass it and the signature to the verifying device. Before the message representative is calculated, they "correct" the flipped bit in the message, ensuring that the signature will now verify.

EXAMPLE 3: An adversary can construct an invalid signature and then target specific instructions to be skipped, such as the subtraction of ct_1 in step 9 of ML-DSA.Verify_internal (NIST FIPS 204 [i.7], algorithm 8), that will result in their invalid signature passing the necessary conditions [i.15].

Each of these examples are illustrative only: enumerating all possible faults and designing individual protections is extremely expensive. As discussed in the general guidance, ETSI TR 104 239-1 [i.2], a simpler mitigation is to repeat steps at risk of faulting and compare the outputs, only proceeding if output agrees. This requires an adversary to accurately induce more than one fault per iteration, which is generally considered extremely difficult.

Reducing an adversary's ability to target a particular instruction by introducing random delays or loop shuffling can limit the effectiveness of fault attacks. Tampering controls can provide an additional level of assurance.

8 Testing and formal verification considerations

8.1 Testing Considerations

The specification of ML-DSA in NIST FIPS 204 [i.7] provides developers with pseudocode for forty-nine different algorithms which together define ML-DSA. Each of these algorithms can be validated using known-answer tests, but to allow for implementation optimizations or side-channel protections NIST does not require implementations to follow the algorithm structure exactly as written in the standard [i.11]. Instead, NIST only provide test vectors [i.12] for each of the three main algorithms: ML-DSA.KeyGen_internal (NIST FIPS 204 [i.7], algorithm 6), ML-DSA.Sign_internal (NIST FIPS 204 [i.7], algorithm 7), and ML-DSA.Verify_internal (NIST FIPS 204 [i.7], algorithm 8).

It is important to be careful when exposing internal algorithms for testing purposes. For example, choosing to store private keys in the seed format as discussed in clause 5.2.1 will require access to ML-DSA.KeyGen_internal to convert the seed into a private key that can be used in the signing algorithm. It is important to test any such algorithm.

As discussed in clause 6.4.2, ML-DSA.Sign_internal uses a rejection sampling loop that can take a variable number of iterations to produce a valid signature. Although NIST FIPS 204 recommends that implementations do not limit the number of iterations, annex C of NIST FIPS 204 [i.7] gives lower bounds for which the different functions will succeed with overwhelming probability. Specific test vectors [i.46] are available to exercise the ML-DSA rejection sampling loop with a probability of failure of 2^{-37}, with plans to extend this work further.

There is more than one way to use the three internal algorithms to construct a signature, as discussed in clause 5.3. This means that additional tests will be required for the wrapper functions, to ensure both pure ML-DSA and HashML-DSA are implemented correctly. Depending on the use-case, some implementations may choose to permit the use of either pure ML-DSA or HashML-DSA but not both. In this case the other variant should not be implemented and therefore tests for this will not be needed.

It is not possible or practical to write an exhaustive list of all the required tests in the present document, since the choice of tests will depend on the threat model and use case of the implementation. However, in addition to the test vectors available from NIST [i.12], supplementary known-answer tests can be written and used to test or fuzz an implementation. For example, additional tests could be written for each of the general implementation considerations outlined in clause 6. It is important that tests are provided for all code paths, including edge cases and exceptions. Additional "unofficial" test vectors are available from a variety of sources such as Project Wycheproof [i.22], Community Cryptography Test Vectors [i.23] or Crucible [i.48].

8.2 Formal Verification Considerations

Formal verification can provide more assurance than testing alone that an implementation of ML-DSA is secure [i.2]. However, formal verification is not binary, instead it is possible to choose the extent to which the implementation is formally verified. This choice needs to be made based on a range of factors, such as the threat model, efficiency requirements, and development resources available for the implementation.

EXAMPLE 1: Cryspen's libcrux [i.27] library provides an implementation of ML-DSA where some critical functions are formally verified using hax and F*. These include field arithmetic and the Number Theoretic Transform (NTT). This library is written in Rust, and therefore also provides static memory-safety. However, researchers have published vulnerabilities in libcrux [i.49] despite the use of formal verification.

EXAMPLE 2: The Post-Quantum Cryptography Alliance maintain mldsa-native [i.42] follows the same methodology as mlkem-native [i.43]. Performance-critical components are written in optimized assembly code which is proven functionally correct using HOL Light, although unlike mlkem-native, mldsa-native does not yet have a complete set of proofs of functional correctness. In the current absence of proofs, these components are partially assured by the use of rigorous testing. The other components are written in C, and these are not proven functionally correct but are proven to be type-safe and memory-safe using the C Bounded Model Checker.

These examples are both at the upper end of what is achievable for a formally verified implementation of ML-DSA. They both rely on use of programming languages and formal verification tools that will require upskilling for most developers to use reliably. For organizations with less resources and expertise it may not be practical to achieve this level of assurance, and instead rigorous testing should be prioritized as the minimum baseline.

Annex A:
NIST FIPS 204 algorithms

A.1 Pure digital signature (external)

- **ML-DSA.KeyGen (algorithm 1).** Returns a key pair or an error.
- **ML-DSA.Sign (algorithm 2).** Takes a private key, message and context string as input. Returns a signature or an error.
- **ML-DSA.Verify (algorithm 3).** Takes a public key, message, signature and context string as input. Returns true or false, or an error.

A.2 Pre-hash digital signature (external)

- **HashML-DSA.Sign (algorithm 4).** Takes a private key, message, context string and pre-hash algorithm as input. Returns a signature or an error.
- **HashML-DSA.Verify (algorithm 5).** Takes a public key, message, signature, context string and pre-hash algorithm as input. Returns true or false.

A.3 De-randomized digital signature (internal only)

- **ML-DSA.KeyGen_internal (algorithm 6).** De-randomized key generation called by ML-DSA.KeyGen. Takes a 32-byte seed as input. Returns a key pair.
- **ML-DSA.Sign_internal (algorithm 7).** De-randomized signing routine called by both ML-DSA.Sign and HashML_DSA.Sign. Takes a private key, formatted message and 32-byte value as input. Returns a signature.
- **ML-DSA.Verify_internal (algorithm 8).** Internal verification routine called by both ML-DSA.Verify and HashML-DSA.Verify. Takes a public key, formatted message and signature as input. Returns true or false.

A.4 Sampling

- **ExpandA (algorithm 32).** Generates the matrix $\hat{A}$ in ML-DSA.KeyGen_internal, ML-DSA.Sign_internal and ML-DSA.Verify_internal. Takes a 32-byte seed as input. Returns a matrix of elements in the NTT domain.
- **ExpandS (algorithm 33).** Generates the private polynomials in ML-DSA.KeyGen_internal. Takes a 64-byte seed as input. Returns a pair of polynomials with coefficients in a bounded range.
- **ExpandMask (algorithm 34).** Generates the ephemeral private vector in ML-DSA.Sign_internal. Takes a 64-byte seed and non-negative integer as input. Returns a vector of polynomials whose coefficients are in a bounded range.
- **SampleInBall (algorithm 29).** Sampling algorithm used to generate the challenge in ML-DSA.Sign_internal and ML-DSA.Verify_internal. Takes a 32-, 48- or 64-byte seed as input. Returns a pseudo-random polynomial with coefficients from $\{-1,0,1\}$ of bounded weight.
- **RejNTTPoly (algorithm 30).** Sampling algorithm used in ExpandA. Takes a 34-byte seed as input. Returns a pseudo-random element in the NTT domain.
- **RejBoundedPoly (algorithm 31).** Sampling algorithm used in ExpandS. Takes a 66-byte seed as input. Returns a pseudo-random polynomial with coefficients in a bounded range.

A.5 Arithmetic

- **NTT (algorithm 41).** Performs the Number Theoretic Transform. Takes a polynomial as input. Returns the corresponding element in the NTT domain.
- **NTT^{-1} (algorithm 42).** Performs the inverse Number Theoretic Transform. Takes an element in the NTT domain as input. Returns the corresponding polynomial.
- **AddNTT (algorithm 44).** Performs addition in the NTT domain. Takes a pair of elements in the NTT domain as input. Returns their sum.
- **MultiplyNTT (algorithm 45).** Performs multiplication in the NTT domain. Takes a pair of elements in the NTT domain as input. Returns their product.
- **AddVectorNTT (algorithm 46).** Performs addition of vectors in the NTT domain. Takes a pair of vectors of elements in the NTT domain as input. Returns the sum of the vectors.
- **ScalarVectorNTT (algorithm 47).** Performs scalar-vector multiplication in the NTT domain. Takes an element in the NTT domain and a vector of elements in the NTT domain as input. Returns the product of the scalar and the vector.
- **MatrixVectorNTT (algorithm 48).** Performs matrix-vector multiplication in the NTT domain. Takes a matrix of elements in the NTT domain and a vector of elements in the NTT domain as input. Returns the product of the matrix and the vector.

A.6 Encoding

- **pkEncode (algorithm 22).** Takes a public key as input. Returns its encoding as a byte string.
- **pkDecode (algorithm 23).** Takes a byte string as input. Returns the public key encoded by the byte string.
- **skEncode (algorithm 24).** Takes a private key as input. Returns its encoding as a byte string.
- **skDecode (algorithm 25).** Takes a byte string as input. Returns the private key encoded by the byte string.
- **sigEncode (algorithm 26).** Takes a signature as input. Returns its encoding as a byte string.
- **sigDecode (algorithm 27).** Takes a byte string as input. Returns the signature encoded by the byte string or an error if the hint field was not encoded correctly.
- **SimpleBitPack (algorithm 16).** Packing algorithm used by pkEncode. Takes an integer b and a polynomial with coefficients in the range $[0, b]$ as input. Returns the encoding of the polynomial as a byte string.
- **SimpleBitUnpack (algorithm 18).** Unpacking algorithm used by pkDecode. Takes an integer b and a byte string as input. Returns a decoding of the byte string as a polynomial with coefficients in the range $[0, b]$.
- **BitPack (algorithm 17).** Packing algorithm used by skEncode and sigEncode. Takes a pair of integers (a, b) and a polynomial with coefficients in the range $[-a, b]$ as input. Returns an encoding the polynomial is a byte string.
- **BitUnpack (algorithm 19).** Unpacking algorithm used by skDecode and sigDecode. Takes a pair of integers (a, b) and a byte string as input. Returns a decoding of the byte string as a polynomial with coefficients in the range $[-a, b]$.
- **HintBitPack (algorithm 20).** Packing algorithm used by sigEncode. Takes a hint polynomial as input. Returns an encoding of the hint polynomial as a byte string.
- **HintBitUnpack (algorithm 21).** Unpacking algorithm used by sigDecode. Takes a byte string as input. Returns a decoding of the byte string as a hint polynomial or an error if the byte string was not encoded correctly.

Annex B:
ML-DSA security properties

B.1 Unforgeability

The main security notions for ML-DSA are Existential Unforgeability against Chosen-Message Attacks (EUF-CMA) and Strong existential Unforgeability against Chosen-Message Attacks (SUF-CMA). These are informally defined as follows:

- **EUF-CMA:** Given a public verification key pk and access to a signing oracle for the corresponding private signing key sk, the adversary cannot construct a valid message-signature pair (m, σ) for pk where m is not the input to an oracle query.
- **SUF-CMA:** Given a public verification key pk and access to a signing oracle for the corresponding private signing key sk, the adversary cannot construct a valid message-signature pair (m, σ) for pk where σ is not the output to an oracle query with input m.

NOTE 1: The public verification keys in the EUF-CMA and SUF-CMA definitions are assumed to be honestly generated. The security properties are not guaranteed to hold for malformed public keys.

SUF-CMA is a stricter security notion than EUF-CMA. The EUF-CMA adversary succeeds if they can forge a valid signature for a new message. The SUF-CMA adversary succeeds if they can forge a valid signature for a new message or for an existing message provided that the signature is new. SUF-CMA security implies EUF-CMA security, but an EUF-CMA secure signature algorithm can fail to be SUF-CMA secure.

EXAMPLE 1: ECDSA [i.6] is not SUF-CMA secure. If $\sigma = (r, s)$ is a valid signature for a message m and public verification key pk, then $\sigma' = (r, -s)$ will also be a valid signature for the same m and pk.

SUF-CMA security can be required by the security proofs for some protocols and SUF-CMA failures have led to issues in some applications.

EXAMPLE 2: The security proofs for SSH in [i.17] and [i.16] require that the signature algorithm is SUF-CMA secure.

NOTE 2: Although the analysis in [i.17] uses the term EUF-CMA, the unforgeability notion defined is SUF-CMA. This was corrected in [i.18].

EXAMPLE 3: The failure of ECDSA to be SUF-CMA secure contributed to transaction malleability issues in Bitcoin [i.29] and was one of the motivations for introducing Schnorr signatures in BIP 340 [i.52].

When implemented correctly, the SUF-CMA security of ML-DSA depends on the hardness of the Module-LWE, Module-SIS, and self-target Module-SIS problems corresponding to the parameter set [i.37], [i.35]. However, implementation vulnerabilities can still cause SUF-CMA or EUF-CMA failures.

EXAMPLE 4: An implementation of ML-DSA key generation that does not use a secure random bit generator can lead to predictable key pairs. This would violate both SUF-CMA and EUF-CMA security.

EXAMPLE 5: An implementation of ML-DSA verification that did not reject longer-than-expected signatures and instead truncated them before use would violate SUF-CMA security, but not EUF-CMA security.

EXAMPLE 6: An implementation of ML-DSA verification that did not perform the hint field encoding checks correctly in HintBitUnpack ([i.7], algorithm 21) would allow an adversary to modify the hint field in a valid signature to give a different signature for the same message that would still be accepted by the implementation. This would violate SUF-CMA security, but not EUF-CMA security.

B.2 Beyond unforgeability

SUF-CMA security prevents one form of malleability: where an adversary attempts to modify a valid signature. Other forms of malleability that involve modifying the message or public key could also be possible without necessarily violating EUF-CMA or SUF-CMA security (see, for example, the discussion in [i.21]).

Cremers et al. [i.26] consider additional security notions for signature algorithms that aim to protect against different forms of malleability. These include Message-Bound Signatures (MBS) and Malicious-Strong Universal Exclusive Ownership (M-S-UEO). They are informally defined as follows:

- **MBS:** The adversary cannot create a pair of distinct messages $m_1 \neq m_2$, a signature σ, and a public verification key pk where the same signature σ is valid for both messages m_1 and m_2 with the same public key pk.
- **M-S-UEO:** The adversary cannot create a pair of messages m_1 and m_2, which need not be distinct, a signature σ, and a pair of distinct public verification keys $pk_1 \neq pk_2$ where the same signature σ is valid both for message m_1 with public key pk_1 and also for message m_2 with public key pk_2.

NOTE 1: Unlike EUF-CMA and SUF-CMA, the public verifications keys in the definitions of MBS and M-S-UEO are chosen by the adversary and do not need to be honestly generated.

EXAMPLE 1: EdDSA [i.4] does not provide MBS as it allows the identity element as a public verification key. An adversary can easily construct a single signature that will be valid for all messages with this public key [i.20].

EXAMPLE 2: EdDSA [i.4] is not M-S-UEO secure as it does not check that public validation keys lie in the correct subgroup. Given a valid message-signature pair (m, σ) for a public key pk, the pair (m, σ) will also be valid for the public key pk' obtained from pk by adding a point of small order [i.20].

NOTE 2: Cremers et al [i.26] also define the new notion of Non Re-signability (NR). However, NR is not directly related to malleability, and it was subsequently discovered that the original definition was flawed [i.31].

MBS and M-S-UEO failures have led to issues in some applications.

EXAMPLE 3: Chosen-prefix collisions on the MD5 hash function when used in the RSA-PKCS#1v1.5 signature algorithm have allowed attackers to generate colliding certificates that share the same signature but contain different identities [i.47]. This is an MBS failure in addition to an EUF-CMA failure.

EXAMPLE 4: The failure of EdDSA to be M-S-UEO secure allowed a double-spend vulnerability in Monero and related digital currencies [i.39].

ML-DSA incorporates the BUFF transform from [i.26] so, when implemented correctly, it will be MBS and M-S-UEO secure. Breaking MBS involves finding a collision in either SHAKE256 or, for HashML-DSA, the pre-hash function. However, implementation vulnerabilities can cause M-S-UEO failures.

EXAMPLE 5: An implementation of ML-DSA verification that truncated longer-than-expected public verification keys instead of rejecting them before use would violate M-S-UEO, but not SUF-CMA security.

EXAMPLE 6: An external μ implementation of ML-DSA verification that allowed an adversary direct access to ML-DSA.ExternalMuVerify (see Table 12) would violate M-S-UEO as the message representative no longer needs to be derived using the public verification key. An adversary could then construct a single message representative μ and signature $\sigma = (\tilde{c}, 0, h)$ that would be valid for any public key of the form $pk = (\rho, 0)$.

Annex C:
Hybrid ML-DSA

C.1 Deployment considerations

It is often recommended that ML-DSA is deployed in a hybrid scheme together with a traditional digital signature algorithm such as ECDSA [i.6] or EdDSA [i.4] in order to mitigate potential ML-DSA implementation vulnerabilities.

While a well-designed hybrid signature scheme will typically preserve EUF-CMA security when implemented correctly, it might not preserve other security properties of ML-DSA.

EXAMPLE 1: A hybrid signature scheme that combines ML-DSA and ECDSA in parallel, such as in [i.41], will not be SUF-CMA secure. Given a valid hybrid signature $\sigma = (\sigma_{\mathrm{MLDSA}}, \sigma_{\mathrm{ECDSA}})$ for a message m with hybrid public key pk, an adversary can modify the ECDSA signature component σ_{ECDSA} to produce a new hybrid signature $\sigma' = (\sigma_{\mathrm{MLDSA}}, \sigma'_{\mathrm{ECDSA}})$ that will be valid for the same m and pk.

EXAMPLE 2: A hybrid signature scheme that combines ML-DSA and EdDSA in parallel, such as in [i.41], will not be M-S-UEO secure. Given a valid hybrid signature σ for a message m with hybrid public key $pk = (pk_{\mathrm{MLDSA}}, pk_{\mathrm{EdDSA}})$, an adversary can modify the EdDSA public key component pk_{EdDSA} to produce a new hybrid public key $pk' = (pk_{\mathrm{MLDSA}}, pk'_{\mathrm{EdDSA}})$ and the signature σ will still be valid for the same message m with pk'.

For further discussion of hybrid deployment considerations, see ETSI TR 103 966 [i.1].

C.2 Implementation considerations

It is important to realize that deploying ML-DSA in a hybrid scheme is not an alternative to implementing it securely. Vulnerabilities in the ML-DSA implementation can still impact the security provided by a well-designed hybrid scheme.

EXAMPLE 1: A side-channel vulnerability in an ML-DSA implementation that allows an adversary to recover the ML-DSA private key will still allow the adversary to recover the ML-DSA component of the hybrid private key when the vulnerable ML-DSA implementation is used in a hybrid scheme.

NOTE: If the ML-DSA component of the hybrid scheme is compromised, then the hybrid is no longer secure against quantum adversaries and cannot be used as a long-term trust anchor.

Similarly, hybrid schemes should still follow the general implementation guidance from ETSI TR 104 239-1 [i.2]:

- Check the format of input hybrid keys and signatures and preform any relevant cryptographic checks on the traditional and ML-DSA components before they are passed to the corresponding component algorithms.
- Use expected formats for output hybrid keys and signatures and perform any relevant consistency checks on the traditional and ML-DSA components before returning the output.
- Zeroise inputs and outputs for the traditional and ML-DSA component algorithms after use and limit direct access to the component algorithms with the relevant components of the hybrid private key.
- Handle errors from the traditional or ML-DSA component algorithms gracefully and avoid leaking sensitive information through errors where necessary.
- Ensure that implementations of the traditional and ML-DSA component algorithms use a secure random bit generator with good source of entropy.

EXAMPLE 2: An implementation of a hybrid scheme that fails to handle input lengths correctly when parsing hybrid keys or signatures can leak information from memory (see, for example, CVE-2024-37305).

EXAMPLE 3: An implementation of a hybrid scheme that fails to reject hybrid signatures where the ECDSA component is $\sigma_{\mathrm{ECDSA}} = (0, 0)$ relies entirely on the security of the ML-DSA component.

Annex D:
Change history

Date	Version	Information about changes
Nov 2025	0.0.1	Initial document. Content added to clauses 5, 6.1, 7 and 8.
Nov 2025	0.0.2	Clause 7 reference updates. Minor editorial updates elsewhere.
Apr 2026	0.0.3	Clause 6.1 expanded. Content added to clauses 6.2 to 6.5. Annexes started.
Apr 2026	0.0.4	Clause 6.1 clarifications. Clause 6.2.1 added.
Jun 2026	0.0.5	Minor updates and editorial tweaks. Completion of annexes.
August 2026	1.1.1	First published version.

History

Version	Date	Status
V1.1.1	September 2026	Publication